Technical Report **Habit & Productivity Analytics API**

Submission Links

- GitHub Repository: https://github.com/X2135/web-service.git
- API Documentation PDF: https://github.com/X2135/web-service/blob/main/docs/API_Documentation.pdf
- Presentation Slides: https://github.com/X2135/web-service/blob/main/docs/Habit%20%26%20Productivity%20Analytics%20API%20%E7%AD%94%E8%BE%A9.pptx
- Deployed Frontend Dashboard: https://web-service-2-8jgm.onrender.com/
- Deployed Backend API: https://web-service-1-iox1.onrender.com

Section 1. Introduction and Project Overview

The Habit & Productivity Analytics API is a data-driven REST API project that transforms raw habit-tracking data into structured, validated, and reusable web services. Built on a 90-day habit dataset, the project focuses on turning noisy real-world CSV data into a relational schema, exposing it through REST endpoints, and generating analytics through a backend summary service.

The system includes full CRUD support for categories and records, JWT-based protection for write operations, an analytics endpoint, a pytest-based validation suite, and a browser dashboard that consumes live API responses. It also supports environment-based database configuration, using SQLite for local development and PostgreSQL for deployment, which improves reproducibility and makes the project suitable for both coursework demonstration and hosted execution.

Section 2. System Architecture and Technology Choices

The system uses a layered architecture to separate responsibilities and improve maintainability. The routing layer defines HTTP endpoints and request handling rules. The CRUD layer contains the main application logic for database operations and analytics aggregation. Schema models define validated request and response contracts, while ORM models represent relational entities in the database. The database layer manages connection setup and session lifecycle.

A typical request flow is: client request → route handler → schema validation → CRUD logic → database read/write → JSON response. This architecture was prioritised to minimise duplicated logic, enforce early validation, and improve testability. The frontend dashboard functions as an API consumer rather than a separate logic layer, which ensures the backend remains the single source of truth for both CRUD data and analytics.

The technology stack was selected to match the needs of a coursework-scale API while still reflecting sound engineering practice. FastAPI was chosen for rapid development and built-in interactive documentation. SQLAlchemy provides structured ORM-based persistence, while the database configuration supports SQLite for local development and PostgreSQL for deployment via `DATABASE_URL`. This keeps local setup simple while supporting persistent hosted environments. Pydantic was chosen to enforce strict validation of inputs and outputs. Pandas was chosen to simplify CSV parsing and preprocessing, and Pytest was chosen to provide repeatable evidence of correctness. These decisions improve consistency, reliability, reproducibility, and maintainability across the system.

Section 3. Data Source, Schema Mapping, and Import Pipeline

This project uses the **"90-Day Habit Tracker for Personal Growth"** dataset obtained from Kaggle(https://www.kaggle.com/datasets/uthaya1995/90-day-habit-tracker-for-personal-growth). It was selected because it contains repeated daily records, mixed habit indicators, and realistic data quality issues such as missing values and inconsistent formats. This is well suited to a data-driven API project, where modelling quality depends directly on preprocessing quality.

The raw CSV data is mapped into two relational entities: habit_categories and habit_records. The first stores category-level metadata, while the second stores dated observations such as habit label, completion status, duration, category reference, and notes. This schema was designed to keep CRUD and analytics queries clear and maintainable.

To support safe deployment, the project also includes a seed tracking table (seed_history) and a guarded pre-deploy seed script. Instead of relying only on row counts, the script records a named seed marker after a successful import. On later deployments, that marker is checked before reseeding. This was designed to improve reliability and prevent duplicate imports.

Raw Field	Internal Field	Transformation Rule
Date	record_date	Parsed as YYYY-MM-DD; invalid rows skipped.
Workout_Duration_Min	duration_minutes	Converted to numeric minutes; non-negative only.
Journaling (Y/N)	activity signal	Normalised as a boolean-style signal.
Reading_Min	activity signal	Converted to a numeric signal.
Sleep_Hours	activity signal	Converted to a numeric signal when available.
activity signals	category	Classified into categories using explicit rules.
multiple fields	completed	Inferred using threshold- and signal-based rules.

Rule-based transformation was chosen as an explicit trade-off against more complex methods. A complex approach might increase modelling flexibility, but explicit rules were prioritised to improve transparency and reproducibility. Instead of opaque heuristics, the project uses mapping logic that can be explained, tested, and justified, which improves maintainability and assessment defensibility.

Section 4. API Design, CRUD Functionality, and Analytics

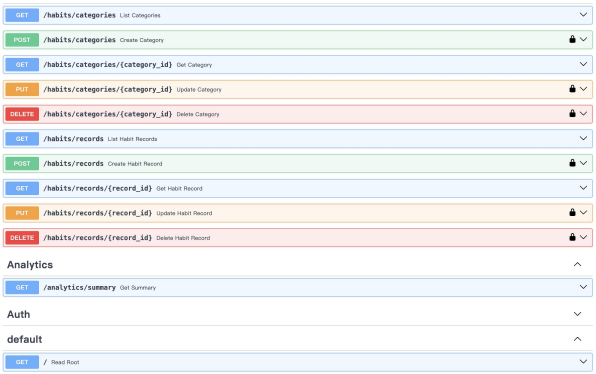
The API is centred on two main resources: habit_categories and habit_records. Each resource supports full CRUD operations, satisfying the coursework requirement for a complete database-backed CRUD model. Categories store grouping metadata, while records store individual observations linked through foreign keys.

The endpoint design follows standard REST conventions. GET retrieves resources, POST creates resources, PUT updates resources, and DELETE removes resources with clear status behaviour, including 204 No Content for successful deletion. Responses are consistently returned as JSON. This was prioritised to ensure predictable behaviour, strong client usability, and stable integration.

Beyond CRUD, the API includes a real aggregation endpoint at /analytics/summary. This endpoint returns six metrics derived from persisted records: total_records, completed_records, completion_rate, average_duration, records_per_category, and daily_trend. These values are computed at the backend layer, which ensures

consistency between storage, business logic, and presentation. The frontend dashboard calls /analytics/summary during initial load and refresh actions. This provides evidence of end-to-end integration between data storage, API processing, and user-facing output.

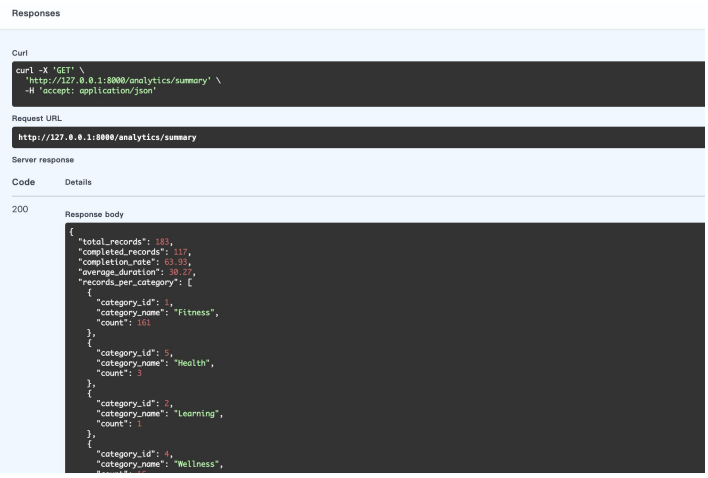
Interactive API documentation (Swagger UI)



Swagger UI

Figure 1. Interactive API documentation (Swagger UI) used for endpoint-level verification.

/analytics/summary response



Analytics summary JSON response

Figure 2. /analytics/summary response returned from the backend aggregation endpoint.

Section 5. Authentication, Error Handling, and Validation

Authentication is implemented using a JWT-based workflow. A client submits credentials to /auth/login (JSON client flow) or /auth/token (OAuth2 form flow for Swagger Authorize), receives a bearer token, and then includes that token in the Authorization: Bearer <token> header for protected requests. In this system, write operations on categories and records (POST, PUT, and DELETE) are protected, while most read operations remain open for demonstration and analysis; the user-context read endpoint /auth/me is protected.

This security model was chosen as a trade-off between **implementation complexity** and **demonstrable security**. It provides meaningful access control while keeping scope appropriate for coursework constraints.

Error handling was designed to be predictable and client-friendly. Business-rule errors or invalid references return 400, authentication failures return 401, missing resources return 404, and validation failures return 422. The API returns errors in a standard detail + code structure, improving consistency for frontend handling and debugging.Validation is enforced through Pydantic schemas before business logic is executed. Required fields, length constraints, and numeric boundaries are checked at the request boundary so malformed data does not propagate into the persistence layer. Together, JWT protection, structured errors, and schema validation improve reliability and maintainability.

Section 6. Testing and Validation

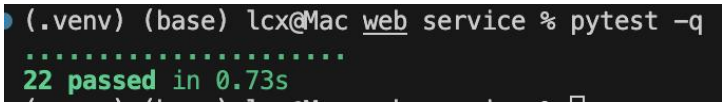
Testing was carried out at the API level using Pytest together with FastAPI's TestClient. The objective was to verify both expected behaviour and failure-path behaviour. The current result is fully passing and provides measurable evidence that the implemented features behave consistently under normal and exceptional conditions.

The test suite covers authentication success and failure, unauthorized access to protected endpoints, CRUD flows for categories and records, duplicate category handling (400), invalid foreign-key references (400), missing resources (404), analytics endpoint behaviour, and schema boundary validation (422).

In addition to automated testing, manual verification was performed using Swagger UI and direct HTTP requests. For example, login functionality was tested by obtaining a JWT token and using it to access protected endpoints such as `POST /habits/records` and `GET /auth/me`. Error scenarios were also manually verified, including submitting invalid `category_id` values (triggering 400 responses) and accessing protected endpoints without a token (triggering 401 responses). The analytics endpoint `/analytics/summary` was tested to confirm that returned values correctly reflect underlying database changes after record creation and updates.

Automated tests were complemented by runtime checks through Swagger UI and helper scripts for API and database verification. This combined strategy improves confidence in both backend correctness and real interactive behaviour. Online verification also passed on the deployed environment, including successful checks of /docs, /analytics/summary, and the end-to-end frontend button flow (login, create, refresh, and delete actions).

Automated test execution result



Pytest terminal output

Figure 3. Automated test execution output showing all tests passed.

Section 7. Challenges and Lessons Learned

A major challenge was source-data quality. The CSV dataset did not map directly to the target schema, so import logic required explicit handling of missing values, type conversion, and rule-based interpretation. This provided evidence that data-driven API reliability depends heavily on transformation quality, not only endpoint implementation.

A second challenge was defining the `completed` field. A duration-only rule would have reduced implementation complexity, but would not represent the mixed dataset faithfully. A multi-signal rule-based approach was chosen to improve explainability while retaining practical accuracy.

A similar decision occurred in analytics placement. Computing summary metrics in the frontend would have been simpler initially, but would have weakened consistency and testability. Moving analytics computation to the backend improved data integrity and maintainability. Overall, this project demonstrates API design as **decision-making under imperfect data**.

Section 8. Limitations and Future Improvements

The current system has clear limitations. Authentication is intentionally lightweight and suitable for coursework demonstration rather than production deployment. User management is minimal, credentials are fixed for demo purposes, and role-based access control is not implemented. The

analytics endpoint remains summary-focused and does not yet include richer filters or comparative views. Although PostgreSQL support exists, versioned schema migration is not yet implemented.

Future work is framed as **production-ready next steps**. Priority upgrades include stronger user management and token controls, richer analytics filtering and comparative reporting, and deployment maturity through migration tooling, observability, and CI/CD enforcement for deployment scripts. These items were intentionally deferred to keep the submitted system focused, explainable, and demonstrable.

Section 9. GenAI Declaration and Reflection

GenAI tools were used in a supporting role rather than as a substitute for engineering judgement. They were used for ideation, debugging support, code refinement suggestions, and documentation drafting.

Tools Used and Purpose (Summary)

- **GitHub Copilot** was used during implementation to accelerate repetitive coding tasks, improve route and function structure, and reduce boilerplate.
- **ChatGPT** was used for design discussion, debugging analysis, and drafting or polishing documentation, including the README, API documentation, technical report, and GenAI declaration.

All key technical decisions were made by me, including schema design, data mapping rules, authentication scope, API structure, deployment logic, and validation strategy. Generated suggestions were accepted only after code-level and runtime validation.

In this project, GenAI improved speed and clarity, but ownership did not shift. **Final decisions remained under human control**, and final responsibility for correctness, coherence, and submission quality remained with me throughout.

Curated GenAI interaction summaries/logs are included as supplementary submission material (appendix) in line with coursework requirements.

Section 10. Appendix Supplementary GenAI Interaction Summaries

- Curated summaries/logs of selected GenAI-assisted interactions used during implementation and documentation refinement.
- Included to support transparency, authorship traceability, and alignment with coursework GenAI disclosure requirements.

Section 11. References

1. Uthaya, S. (n.d.) *90-Day Habit Tracker for Personal Growth*. Kaggle. Available at: https://www.kaggle.com/datasets/uthaya1995/90-day-habit-tracker-for-personal-growth (Accessed: 4 April 2026).
2. FastAPI (n.d.) *FastAPI Documentation*. Available at: https://fastapi.tiangolo.com/ (Accessed: 20 April 2026).
3. SQLAlchemy (n.d.) *SQLAlchemy 2.0 Documentation*. Available at: https://docs.sqlalchemy.org/ (Accessed: 13 April 2026).
4. Pydantic (n.d.) *Pydantic Documentation*. Available at: https://docs.pydantic.dev/ (Accessed: 20 April 2026).
5. pytest (n.d.) *pytest Documentation*. Available at: https://docs.pytest.org/ (Accessed: 20 April 2026).
6. IETF (2015) *RFC 7519: JSON Web Token (JWT)*. Available at: https://datatracker.ietf.org/doc/html/rfc7519 (Accessed: 20 April 2026).